EO Data Pipeline – Test Suite Documentation

MED Consortium · EOHA Platform

Milestone 2 · Verified: May 2025

Document Purpose

This document specifies the automated test suite written for the `eo_datapipeline` module – the monthly Earth Observation ingestion pipeline that underpins the EOHA (Earth Observational Health Analytics) platform. The pipeline fetches satellite and reanalysis data from Google Earth Engine (GEE), exports per-ward statistics to Google Cloud Storage (GCS), and upserts the results into a PostgreSQL database.

The test suite provides confidence that:

- GEE image composition logic behaves correctly without live credentials.
- DataFrame transform functions produce well-typed, null-safe output that matches the database schema.
- PostgreSQL upsert SQL is structurally correct and idempotent.
- The full pipeline flow – from task polling through to DB write – handles both happy-path and failure scenarios.

Pipeline Architecture Summary

#	Step	Description
0	Authentication	Service account JSON loaded from environment; GEE and GCS clients initialised.
1	Month Resolution	Previous calendar month derived as the processing window.
2	Asset Loading	South African ward boundaries, WorldPop population, and ESA land cover images loaded.
3	Image Composition	Per-province: LST, NDVI, NDWI, ERA5 temp and soil moisture stacked into a single multi-band image.
4	Region Reduction	Mean and mode reducers applied at 1 000 m scale over ward polygons.
5	Export	One GEE batch export task per province; CSV written to GCS.
6	Task Polling	Active tasks polled every 30 seconds; pipeline exits on any failure.
7	Transform	GCS CSV parsed; zone and observation DataFrames constructed and cleaned.
8	DB Upsert	Both tables upserted to PostgreSQL using `ON CONFLICT DO UPDATE`.

Test Suite Structure

The test suite is organised into four modules, each targeting a distinct architectural layer. All tests run with `pytest` and require no GCP credentials.

Module	Layer	Tests
`test_unit_gee.py`	GEE composition (unit)	13
`test_integration_transforms.py`	GCS → DataFrame (integration)	27
`test_integration_db.py`	PostgreSQL upsert (integration)	23
`test_e2e_pipeline.py`	Full pipeline flow (E2E)	35
Total		**98**

Shared Fixtures (`conftest.py`)

All fixtures are defined in `tests/conftest.py` and are available across every module. The canonical two-row sample DataFrame mirrors the CSV format exported by GEE for Limpopo wards.

Fixture	Purpose
`sample_df`	Clean two-row GEE export DataFrame (Ward 1 and 2, Musina).
`sample_df_with_nulls`	Same data with deliberate null fields for null-handling tests.
`sample_df_bad_ward_id`	One row with a non-numeric `WardID` to test `dropna` paths.
`sample_df_duplicate_ward`	Two rows with the same `WardID` to test deduplication.
`mock_gcs_client`	`MagicMock` GCS client whose `blob.download_as_text()` returns the sample CSV.
`mock_gcs_client_empty`	GCS client returning a header-only CSV (empty result).
`mock_db_cursor`	`MagicMock` psycopg2 cursor.
`mock_ee_task_success`	EE task mock: `active()` False, state `COMPLETED`.
`mock_ee_task_failure`	EE task mock: `active()` False, state `FAILED`.
`mock_ee_task_slow`	EE task mock: `active()` returns True, True, False (two polls).

Unit Tests: GEE Composition Layer

Overview

These tests verify the Google Earth Engine image composition logic without any network calls. The `ee` module is replaced by a `MagicMock` namespace whose chaining API (e.g. `filterDate().select().mean().re`

is fully stubbed. `ee.Algorithms.If` is configured via `side_effect` to return either the *with-data* or *without-data* branch on demand.

TestBuildSafeBandImage

#	Test Name	Assertion
1	`test_returns_with_data_branch`	Result is not None when collection is non-empty.
2	`test_select_called_with_correct_band`	`collection.select()` receives the correct band name.
3	`test_mean_is_called_on_filtered_coll`	`.mean()` is invoked after `.select()`.
4	`test_scale_factor_applied`	`.multiply(scale)` called on the mean image.
5	`test_offset_applied`	`.add(offset)` called after multiply.
6	`test_output_band_renamed`	`.rename(output_name)` called on the result image.
7	`test_fallback_branch_uses_selfmask`	When no data, `selfMask()` is invoked on the empty image.

TestProcessProvince

#	Test Name	Assertion
1	`test_filter_called_with_province_nam`	`ee.Filter.eq('Province', name)` used to subset wards.
2	`test_reduce_regions_uses_mean_reducer`	`reduceRegions()` called with `ee.Reducer.mean()`.
3	`test_reduce_regions_scale_is_1000`	Scale parameter equals 1 000 metres.
4	`test_habitat_mode_reducer_fires`	Separate `reduceRegions` call with `ee.Reducer.mode()`.
5	`test_population_density_multiplied_b`	WorldPop per-pixel count multiplied by 100 to get per-km^2.
6	`test_month_label_set_on_features`	`feature.set('Month', label)` applied via `.map()`.

Integration Tests: GCS and DataFrame Transforms

Overview

These tests exercise `build_zone_dataframe`, `build_observation_dataframe`, and `load_csv_from_gcs` using real pandas operations on in-memory data. No GCS credentials are required; the storage client is injected as a `MagicMock`.

TestBuildZoneDataframe (15 tests)

Key assertions grouped by concern:

Concern	Tests
Schema	Correct column set; `zone_type = 'Ward'` for all rows.
Type safety	`zone_id` integer; `latitude/longitude` float; `base_population` integer; `external_zone_id` string.
Null handling	NaN/None inputs produce NaN or None in output (accepted as SQL NULL by psycopg2); `Population_Count` null defaults to 0 via `fillna(0)`.
Data integrity	Province and municipality names pass through unchanged.
Edge cases	Non-numeric `WardID` row is dropped; duplicate `WardID` rows are deduplicated to one row.

TestBuildObservationDataframe (11 tests)

Concern	Tests
Schema	Eight required columns present.
Date logic	`date_recorded` is `datetime64`; day is always 1; year and month match the month label.
Value correctness	LST and NDVI values match the fixture to six decimal places.
Null handling	Null LST input produces NaN or None output.
Scale test	Nine-province concatenation yields exactly 18 rows.

TestLoadCsvFromGcs (8 tests)

Concern	Tests
Return type	Function returns a `pd.DataFrame`.
GCS API calls	`bucket(name)` and `blob('pipeline_output/{filename}.csv')` called with exact arguments.
Content	Returned DataFrame has expected columns and row count.
Empty CSV	Header-only CSV returns an empty DataFrame (no crash).
All provinces	All 9 slugified province filenames resolve correctly.

Integration Tests: PostgreSQL Upsert Logic

Overview

The `execute_batch` function (psycopg2) is patched at the module level so no live database connection is required. Tests inspect the SQL string, the row tuples, and keyword arguments passed to the mock.

TestUpsertZoneRows (10 tests)

Concern	Tests
SQL correctness	`INSERT INTO zone` present; `ON CONFLICT (zone_id) DO UPDATE SET` present; `population_density_per_km2` in UPDATE clause.
Row structure	Each row is a tuple; every tuple has exactly 11 fields; first field is an integer (zone_id).
API contract	`execute_batch` called exactly once; cursor passed as first argument.
Return value	Row count equals the number of rows in the input DataFrame.
Edge case	Empty DataFrame produces 0-count return without error.

TestUpsertObservationRows (10 tests)

Concern	Tests
SQL correctness	`INSERT INTO malaria_observation`; conflict key is `(zone_id, date_recorded)`; all six environmental indicator columns in UPDATE clause.
Row structure	Each row is a tuple with exactly 8 fields.
Performance	`page_size=1000` keyword argument passed to `execute_batch`.
Edge case	Empty DataFrame returns 0 without error.

TestDbErrorHandling (2 tests)

Verifies that exceptions raised inside `execute_batch` propagate upward to the caller (the pipeline loop), which is responsible for calling `connection.rollback()` and `sys.exit(1)`.

End-to-End Tests: Full Pipeline Flow

Overview

The E2E layer wires all components together using mocked external surfaces. No credentials, network calls, or real database connections are used. Thirty-five tests cover the following scenarios.

TestPollTaskUntilComplete (7 tests)

Scenario	Expected Outcome
Task completes immediately	Returns `True`.
Task fails	Returns `False` (no exception raised).
Task active for two polls	`active()` called three times; sleep called twice.
Poll interval respected	`time.sleep()` called with the configured interval.
Status check after inactive	`task.status()` invoked after `active()` returns False.
CANCELLED state	Returns `False`.
UNSUBMITTED state	Returns `False`.

TestPipelineHappyPath (9 tests)

Exercises the full chain: `load_csv_from_gcs` → `build_zone_dataframe` → `build_observation_dataframe` → `upsert_zone_rows` → `upsert_observation_rows`.

Key assertions: zone and observation row counts are equal and match the fixture; zone IDs propagate identically through both DataFrames; GCS bucket and blob paths are called with exact string arguments; date fields are the first day of the correct month; no null zone IDs appear in either output.

TestAllProvincesProcessed (6 tests)

Confirms that the `PROVINCES` constant contains exactly nine entries, includes Limpopo and Gauteng, and that slugified filenames contain no spaces. A loop test simulates nine sequential GCS reads and verifies that each returns a valid zone and observation DataFrame.

TestGcsFailurePath (2 tests)

A `GCS 403 Forbidden` exception from `download_as_text()` propagates unmodified, and a missing blob also raises as expected.

TestTaskFailurePath (3 tests)

Tasks in states `FAILED`, `CANCELLED`, and `UNSUBMITTED` all cause `poll_task_until_complete` to return `False`.

Test Execution

Prerequisites

```
pip install pytest pytest-mock pandas psycopg2-binary
```

Listing 1: Install dependencies

No GCP credentials, GEE project access, or live database are required to run the suite.

Running All Tests

```
cd eoha_tests
pytest tests/test_unit_gee.py \
       tests/test_integration_transforms.py \
       tests/test_integration_db.py \
       tests/test_e2e_pipeline.py \
       -v --tb=short
```

Listing 2: Run full suite

Running a Single Layer

```
pytest tests/test_unit_gee.py -v
```

Listing 3: Run unit tests only

Expected Output

```
========================== 98 passed in 0.82s ==========================
```

Listing 4: Passing run summary

Results Summary

Module	Total	Passed	Status
`test_unit_gee.py`	13	13	PASS
`test_integration_transforms.py`	27	27	PASS
`test_integration_db.py`	23	23	PASS
`test_e2e_pipeline.py`	35	35	PASS
Total	**98**	**98**	PASS

Design Decisions and Constraints

No live GEE calls. The `ee` module is mocked entirely. GEE's deferred computation model means that unit tests cannot verify the actual pixel values returned by the API – they can only verify that the correct API methods are called in the correct order with the correct arguments.

No live psycopg2 connection. `execute_batch` is patched at the module boundary. SQL correctness is verified by string inspection; a future integration stage should run against a containerised Postgres instance (e.g. via `pytest-postgresql` or Docker Compose).

Null handling. Python's `pandas.DataFrame.where(pd.notnull())` converts NaN to NaN (not Python `None`) in float columns. psycopg2 treats both as SQL `NULL`, so both representations are accepted in the test assertions.

Extractable module. The original pipeline is a top-level script that calls `ee.Initialize` and `storage.Client` at import time. A refactored `eo_datapipeline.py` module was created that exposes all business logic as pure functions, enabling testability without any environment variables.

Recommended Next Steps

1. **Containerised DB tests.** Add a `pytest-postgresql` fixture to run the upsert SQL against a real Postgres schema, verifying the `ON CONFLICT` behaviour at the database level.
2. **GEE acceptance tests.** Add an optional test class gated behind a `-live-gee` flag that authenticates with a test GCP project and submits a single-ward export for one province.
3. **Schema drift tests.** Add a test that loads the real `CREATE TABLE` SQL and verifies the column names and types match the DataFrame column set produced by the transforms.
4. **CI integration.** Wire the suite into the Cloud Run deployment pipeline so that any commit that breaks a transform or upsert is caught before the container image is pushed.